

Лабораторна робота № 5

Тема: Основи роботи з **Git**. Налаштування **Git**. Робота з комітами.

Мета роботи: навчитися налаштовувати Git, створювати файли, комітити їх та переглядати коміти.

Теоретичні відомості

Для виконання лабораторної роботи в командному рядку необхідно виконати послідовність таких кроків.

1. Налаштування Git.

Налаштування системи Git передбачає, в першу чергу, вказати ім'я користувача та e-mail, які використовуються для підпису комітів і відправлення змін в віддалений репозиторій.

У Git існує три місця, де зберігаються настройки:

- на рівні системи;
- на рівні користувача;
- на рівні проекту (репозиторію).

Для того, щоб настроїти Git на тому чи іншому рівні треба змінити безпосередньо конфігураційні файли, але для цього потрібно знати їх формат, або скористатися спеціальними командами, які надає Git.

Налаштування Git за допомогою утіліти командного рядка є найбільш зручним та безпечним способом. Незалежно від того, на якому рівні змінюють налаштування, команда починається **git config**.

Для рівня системи, треба написати:

```
> git config --system  
    рівня користувача:  
> git config --global  
    рівня програми:  
> git config
```

Після цієї команди вказують параметр та його значення.

Наприклад, встановимо ім'я та адресу електронної пошти користувача. Кожен коміт в git використовує цю інформацію, і вона незмінно включена у коміти, які ви робите:

```
> git config --global user.name "User"  
  
> git config --global user.email "user@company.com"
```

Для перегляду введених змін скористаємось командою:

```
> git config --list
```

При потребі можна перевірити значення конкретного ключа, виконавши

команду `git config <key>`. Наприклад,

```
$ git config user.name
```

Коли ім'я вже вказано, можна налаштувати текстовий редактор за замовчуванням, який буде використовуватися Git при необхідності ввести повідомлення. Якщо це не налаштовано, Git використовує типовий системний редактор.

Для цього треба модифікувати параметр **core.editor**:

- варіант для *Linux*:

```
> git config --global core.editor "nano"
```

- варіант для *Windows*:

```
> git config --global core.editor "notepad.exe"
```

Для отримання допомоги при використанні Git треба виконати команду:

```
$ git help config
```

Крім того, при потребі скороченого переліку можливостей ключів достатньо набрати потрібну команду з використанням опції `-h`, наприклад,

```
$ git add -h
usage: git add [<options>] [--] <pathspec>...

-n, --dry-run          dry run
-v, --verbose          be verbose

-i, --interactive      interactive picking
-p, --patch            select hunks interactively
-e, --edit             edit current diff and apply
-f, --force            allow adding otherwise ignored files
-u, --update           update tracked files
-N, --intent-to-add    record only the fact that the path will be added later
-A, --all              add changes from all tracked and untracked files
--ignore-removal       ignore paths removed in the working tree (same as --no-all)
--refresh              don't add, only refresh the index
--ignore-errors        just skip files which cannot be added because of errors
--ignore-missing       check if - even missing - files are ignored in dry run
--chmod <(+/-)x>      override the executable bit of the listed files
```

2. Створення Git-репозиторію.

Зазвичай Git-репозиторій створюють одним з двох способів:

- 1) Беруть локальну директорію, яка не знаходиться під контролем версій та перетворюють її на сховище Git;
- 2) Клонують існуючий Git-репозиторій з іншого сховища.

У будь-якому разі на локальній машині буде знаходитись готове до роботи Git сховище.

Для того щоб створити репозиторій спершу треба створити папку, в якій

він буде розташовуватися. У нашому випадку це буде каталог з назвою **repo**.

```
> mkdir repo
```

Тепер перейдемо в цей каталог

```
> mkdir repo
```

Створимо в ньому пустий репозиторій

```
> git init
```

При отриманні копії існуючого Git репозиторію — наприклад, проекту, в якому ви хочете прийняти участь — потрібно набрати команду `git clone <url>`. Наприклад, при клонуванні бібліотеки Git `libgit2`, треба виконати команду:

```
$ git clone https://github.com/libgit2/libgit2
```

Це створить директорію під назвою `libgit2`, проведе ініціалізацію директорії `.git`, забере всі дані для репозиторію, та приведе директорію до стану останньої версії.

3. Створення першого коміта.

Тепер у вас на локальній машині має бути справжній Git репозиторій та робочий каталог з усіма файлами цього проекту. Зазвичай, при зміні файлів у проекті, виникає потреба зберегти зміни у репозиторії.

Кожен файл робочої директорії може бути в одному з двох стнів: контрольований (`tracked`) чи неконтрольований (`untracked`). Контрольовані файли — це файли, що були в останньому знімку. Вони можуть бути не зміненими, зміненими або індексованими. Якщо коротко, контрольовані файли — це файли, про які Git щось знає.

Неконтрольовані файли — це все інше, будь-які файли у вашій робочій директорії, що не були у вашому останньому знімку та не існують у вашому індексі. Якщо ви щойно зробили клон репозиторія, усі ваші файли контрольовані та не змінені, адже Git щойно їх отримав, а ви нічого не редагували.

По мірі редагування файлів, Git бачить, що вони змінені, адже ви їх змінили після останнього коміту. Впродовж роботи ви вибірково індексуєте ці змінені файли та потім зберігаєте всі індексовані зміни, та цей цикл повторюється.

Перевірка списку комітів, які були відправлені в репозиторій:

```
> git log
```

```
fatal: your current branch 'master' does not have any commits yet
```

Нам показує, що такий список пустий, адже ми нічого туди не відправляли.

Перевірка статусу робочого каталогу:

```
> git status
```

Створення в поточному каталозі пустого файлу.

```
> touch README.md
```

При виконанні команди `git status`, побачимо, що в нашому каталозі з'явився один неконтрольований файл: `README.md`.

```
> git status
On branch master

Initial commit

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    README.md

nothing added to commit but untracked files present (use "git add" to track)
```

Щоб почати контролювати новий файл, потрібно використати команду:

```
$ git add README.md
```

Виконаємо `git status` для того, щоб подивитися на те, що зараз відбувається в нашому каталозі.

```
> git status
On branch master

Initial commit

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

    new file:   README.md
```

Видно, що в кеш було додано один файл з іменем `README.md` і тепер представлений набір файлів готовий до відправлення в репозиторій, тобто до коміту.

```
> git commit -m "[create repository]"
[master (root-commit) 500067c] [create repository]
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md
```

Перевіримо статус каталогу:

```
> git status
On branch master
```

```
nothing to commit, working tree clean
```

Як видно з моменту останнього коміту ніяких змін в робочому каталозі не проводилося.

Тепер поглянемо на список комітів.

```
> git log
commit 500067cc0b80643d38e2a24e9e0699031ada6be3
Author: Writer <writer@someserver.com>
Date:   Mon Feb 12 22:51:14 2018 +0500

[create repository]
```

З наведеної інформації видно, що був відправлений один коміт, який має ID: 500067cc0b80643d38e2a24e9e0699031ada6be3. Автор даного коміту Writer, він (коміт) був створений Mon Feb 12 22:51:14 2018 +0500, з повідомленням: [create repository]. Це досить докладна інформація, коли комітів стане багато, такий формат виведення буде не дуже зручним, скорочений варіант виглядає так.

```
> git log --oneline
500067c [create repository]
```

Отже, основні команди:

Створення порожнього сховища.

```
> Git init
```

Додавання файлів в stage (кеш).

```
> Git add filename
```

Створення комітів.

```
> Git commit -m "message"
```

Перегляд статусу каталогу.

```
> Git status
```

Перегляд комітів в репозиторії.

```
> Git log
```

Перегляд коммітів в репозиторії зі скороченим виведенням інформації.

```
> Git log --oneline
```

4. Перегляд інформації по комітам.

Внесемо ще кілька змін в наш репозиторій. Додамо текст в файл README.md.

```
> echo "Project 51"> README.md
```

Зафіксуємо ці зміни в репозиторії.

```
> git add.
```

```
> git commit -m "[add]: caption into README file"
```

Створимо файл main.c і додамо його в репозиторій.

```
> touch main.c
> git add.
> git commit -m "[create]: main file of program"
```

Таким чином в нашому репозиторії вже має бути три коміти, перевіримо це.

```
> git log
commit 2b826bb4929fb1c8166ef05b540ce2cc68f3ebb2
Author: Writer <writer@somecompany.com>
Date:   Mon Mar 5 23:17:08 2018 +0500

    [create]: main file of program

commit bc067c88c427dbedbb02817f9ae25241dcae4d07
Author: Writer <writer@somecompany.com>
Date:   Mon Mar 5 23:15:12 2018 +0500

    [add]: caption into README file

commit a98cce47b59256d00a853c421af4f7b9f0dc0a29
Author: Writer <writer@somecompany.com>
Date:   Mon Mar 5 23:10:51 2018 +0500

    [create repository]
```

Коміти зберігаються від нових до старих. Зробимо ще кілька змін.

```
> touch main.h
> git add .
> git commit -m "[create]: header for main"
> touch .gitignore
> git add .
> git commit -m "[create]: git ignore file"
> echo "*.tmp" > .gitignore
> git add .
> git commit -m "[add] ignore .tmp files">
```

Знову отримаємо список всіх комітів.

```
> git log
commit cf3d9d8f7b283267a085986e85cc8f152cca420d
Author: Writer <writer@somecompany.com>
Date:   Mon Mar 5 23:21:59 2018 +0500

    [add] ignore .tmp files

commit a7b88eed6110b6ebb1fc4d96f4399e4cbb8339e7
Author: Writer <writer@somecompany.com>
Date:   Mon Mar 5 23:21:11 2018 +0500
```

```
[create]: git ignore file
```

```
commit c185b80ca916af7d6f068450f6cafb073d955c40
Author: Writer <writer@somecompany.com>
Date:   Mon Mar 5 23:20:26 2018 +0500
```

```
[create]: header for main
```

```
commit 2b826bb4929fb1c8166ef05b540ce2cc68f3ebb2
Author: Writer <writer@somecompany.com>
Date:   Mon Mar 5 23:17:08 2018 +0500
```

```
[create]: main file of program
```

```
commit bc067c88c427dbedbb02817f9ae25241dcae4d07
Author: Writer <writer@somecompany.com>
Date:   Mon Mar 5 23:15:12 2018 +0500
```

```
[add]: caption into README file
```

```
commit a98cce47b59256d00a853c421af4f7b9f0dc0a29
Author: Writer <writer@somecompany.com>
Date:   Mon Mar 5 23:10:51 2018 +0500
```

```
[create repository]
```

Кількість комітів в репозиторії вже таке, що переглядати інформацію про них в тому вигляді, в якому видає git log вже незручно. Для того, щоб скоротити кількість потрібної інформації можна скористатися ключем `-oneline`, при цьому буде виведено частину ідентифікатора і повідомлення комітів.

```
> git log --oneline
cf3d9d8 [add] ignore .tmp files
a7b88ee [create]: git ignore file
c185b80 [create]: header for main
2b826bb [create]: main file of program
bc067c8 [add]: caption into README file
a98cce4 [create repository]
```

У такому вигляді працювати з комітами вже набагато зручніше. При перегляді `n` останніх комітів, треба вказати їхню кількість після ключа `-n`.

```
> git log -n 3 --oneline
cf3d9d8 [add] ignore .tmp files
a7b88ee [create]: git ignore file
c185b80 [create]: header for main
```

5. Додавання та видалення файлів в репозиторії.

Додавання файлів в репозиторій - це досить проста операція, мало чим відрізняється від відправки змін у контрольованих файлах в репозиторій. Створимо новий репозиторій, для цього перейдіть в каталог, в якому ви хочете його розташувати і введіть команду `git init`.

```
> git init
```

Створіть в каталозі файл `README.md`.

```
> touch README.md
```

Перевірте стан поточного каталогу.

```
> git status
```

Для того, щоб створений файл почати контролювати. То треба виконати команду:

```
> git add README.md
```

Після цього ще раз перевірити стан:

```
> git status
```

```
On branch master
```

```
Initial commit
```

```
Changes to be committed:
```

```
(use "git rm --cached <file>..." to unstage)
```

```
new file:   README.md
```

Видно, що інформація про новий файл попала в кеш. Для того, щоб це зафіксувалось в репозиторії треба:

```
> git commit -m "add README.md file"
```

```
[master (root-commit) 0bb6c94] add README.md file
```

```
1 file changed, 0 insertions(+), 0 deletions(-)
```

```
create mode 100644 README.md
```

Тепер в робочому каталозі і в кеші немає об'єктів, інформацію про зміну яких необхідно внести в репозиторій.

Видалення файла з кеш.

Створимо ще один файл.

```
> touch main.c
```

"Відправимо" файл `main.c` в кеш.

```
> git add main.c
```

Внесемо зміни в `README.md`.

```
> echo "# readme"> readme.md
```

Інформацію про це також відправимо в кеш.

```
> git add readme.md
```


Подивимося на стан stage.

```
> git status
on branch master
changes to be committed:
  (use "git reset head <file> ..." to unstage)
```

```
    modified: readme.md
    new file:   main.c
```

Якщо нам необхідно прибрати з stage, якийсь із цих файлів (main.c або README.md), то для цього можна скористатися командою `git -rm cached <filename>`, зробимо це для файлу main.c.

```
> git rm --cached main.c
rm 'main.c'
```

Тепер подивимося на стан робочої директорії і stage.

```
> git status
on branch master
changes to be committed:
  (use "git reset head <file> ..." to unstage)
```

```
    modified: readme.md
```

```
untracked files:
  (use "git add <file> ..." to include in what will be
committed)
```

```
    main.c
```

Видно, що зміни в файлі README.md готові для комітів, а ось файл main.c перейшов в стан - неконтрольованих. Відправимо main.c в stage і, після цього, зробимо коміт в репозиторій.

```
> git add main.c
> git commit -m "add main.c and do some changes in readme.md"
[master 49049bc] add main.c and do some changes in readme.md
 2 files changed, 1 insertion (+)
 create mode 100644 main.c
```

Видалення файлів з git сховища

Видалити файл зі сховищ можна двома способами: перший - видалити його з робочої директорії і повідомити про це git; другий - скористатися засобами git. Почнемо з першого способу. Для початку подивимося, які файли у нас зберігаються в репозиторії.

```
> git ls-tree master
100644 blob 7e59600739c96546163833214c36459e324bad0a    README.md
```

```
100644 blob e69de29bb2d1d6434b8b29ae775ad8c2e48c5391    main.c
```

Видалимо файл `main.c` з робочої директорії.

```
> rm main.c
> ls
README.md
```

Повідомимо про це систему `git`.

```
> git rm main.c
rm 'main.c'
```

Замість команди `git rm` можна використовувати `git add`, але саме слово `add` в даному випадку буде звучати дещо неоднозначно, тому краще використовувати `rm`. На даному етапі ще можна повернути все назад за допомогою команди `git checkout - <filename>`, в результаті, в нашу робочу директорію буде скопійований файл зі сховищ. Створимо коміт, що фіксує видалення файлу.

```
> git commit -m "remove main.c"
[master d4e22ae] remove main.c
1 file changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 main.c
```

Тепер в репозиторії залишився тільки один файл `README.md`.

```
> git ls-tree master
100644 blob 7e59600739c96546163833214c36459e324bad0a    README.md
```

Другий спосіб - це відразу використовувати команду `git rm` без попереднього видалення файлу з директорії.

```
> git rm main.c
rm 'main.c'

> git commit -m "deleted: main.c file"
[master ba7d027] deleted: main.c file
1 file changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 main.c
```

Посібник з використання `Git` українською мовою знаходиться за посиланням

<https://tinyurl.com/yc7tmjh6>

Порядок виконання роботи:

1. Завантажити *git* клієнт та встановити його.
2. Створити папку, в якій буде записано проект.
3. Провести ініціалізацію репозиторію в даній папці.
4. Встановити налаштування імені та пошти.

5. Створити в папці текстовий файл *file1.txt* , в якому записати свої анкетні дані і проіндексувати його.
6. Виконати перший коміт.
7. Відкрити файл *file1.txt* і додати до нього рядок, в якому дописати свій номер телефону. Проіндексувати зміни.
8. Створити новий файл *file2.txt*, в якому записати прізвища та ініціали свої колег по команді. Проіндексувати зміни.
9. Зробити 2-й коміт.
10. Переіменувати файл *file1.txt*, зафіксувати зміни.
11. Створити ще один файл, який буде містити об'єднану інформацію перших 2-х файлів.
12. Проіндексувати його з коментарем «об'єднано 2 файли».
13. Видали 1-й файл. Зробити коміт і коментар.
14. Вивчити можливості команди **git log** і виконати різні варіанти виводу інформації і її відбору:
 - вивести всі коміти, з інформацією про те, які файли були змінені;
 - вивести інформацію про 1-й коміт;
 - вивести останніх 3 коміти в форматі: автор, коментар;
 - вивести коміти, в яких відбувались зміни 3-го файлу.

Примітка: Назви файлів *file2.txt* та *file2.txt* мають бути змінені відповідно до власного прізвища.

Шпаргалка Git

Задача	Команда
Створити репозиторій	git init
Перевірити статус	git status
Додати файл у staging	git add <file>
Додати всі зміни	git add .
Зробити коміт	git commit -m "message"
Переглянути історію	git log
Скорочений log	git log --oneline
Видалити файл	git rm <file>